

Student Assignment Guide

IFT 458 Level 2 Resource Authentication

Use the Game Score Tracker API to demonstrate that every game and score operation requires authentication. Configure the supplied project, register and log in, test access with and without a bearer token, and submit evidence from your own run.

Student name _____ Student ID _____

What you will learn

- Distinguish password verification, a reusable session, and authorization rules.
- Use Swagger to send JSON requests and read actual HTTP responses.
- Verify authenticated CRUD for games and scores and reject anonymous requests.
- Explain password hashing, token expiry, and the limits of authentication using your logs.

What you need

Have the supplied IFT_Level2-Authentication project, Node.js and npm compatible with its dependencies, a code editor, Chrome or another browser, a working MongoDB database, and Word or another DOCX editor. Check `node --version` and `npm --version` before installing dependencies. Use the instructor-provided database setup if Atlas is not already configured.

How this assignment works

This is a configuration, testing, and explanation assignment. The authentication implementation is already provided. Complete the tasks through Swagger; change only `.env` or `config.env` unless the instructor explicitly assigns a coding extension. Use your own isolated coursework database and test records.

The original Level 1 handout and some startup messages describe anonymous access. For this Level 2 exercise, a game or score request without a valid token must return 401. Do not remove authentication to reproduce the earlier behavior. Roles and ownership restrictions are not implemented in this build.

Your submission

Prepare one Word report with numbered screenshots, a completed results matrix, and a 90 to 120 word Security Log Analysis. Include the timestamped application log and package both files in a ZIP. Use the due date and submission location provided by your instructor in Canvas.

1 Set up and start the project

1. Open IFT_Level2-Authentication in your editor. Open a terminal in the folder containing package.json and server.js. Run npm install and resolve any installation errors before continuing.
2. Copy config.env.example to .env only if .env does not already exist. Add your own settings. Do not overwrite a working configuration. The application loads .env first; config.env supplies values that are not already set.

```
NODE_ENV=development
PORT=4001
API_VERSION=/api/v1
DATABASE=mongodb://127.0.0.1:27017/ift458_level2
STUDENT_NAME=Your Full Name
STUDENT_ID=Your Student ID
```

The local example requires MongoDB to be running on your computer. For Atlas, replace DATABASE with the template below and fill the variables using your own database credentials. Replace <your-cluster> with your actual cluster host.

```
DATABASE=mongodb+srv://<db_username>:<db_password>@<your-cluster>/<db_name>
DATABASE_USER=your_database_user
DATABASE_PWD=your_database_password
DATABASE_NAME=ift458_level2
```

Use the exact variable names shown for this build. An Atlas database user is different from an application user registered through /users/signup. Never place real database credentials in screenshots or your submitted report.

3. If the instructor requires sample records, run npm run seed once in your isolated coursework database. This command deletes existing users and games before importing sample data. Skip seeding if you need to preserve those records; this assignment lets you create your own through the API.
4. Run npm test in the project terminal. Record the pass and fail totals. The tests use a temporary MongoDB database; they do not replace the manual evidence required below.
5. Run npm start. Keep this terminal open. Confirm that the server connects to MongoDB and that the startup log identifies you. Open <http://localhost:4001/api-docs>. If PORT was changed, use that port consistently.
6. Confirm the Swagger title includes Level 2 Resource Authentication. Restart the server after configuration or code changes. Avoid using an older deployed Level 1 URL for this assignment.

Evidence 1

Capture the startup success and your student identity without revealing credentials. Capture the automated test summary separately. If setup fails, record the exact error, command, and troubleshooting steps rather than claiming success.

2 Register and obtain a session

1. In Swagger, expand POST /users/signup. Select Try it out, enter a unique coursework email and the JSON below, and select Execute. Read Server response. Expect 201 Created. Signup does not issue a token.

```
{
  "name": "Student One",
  "email": "student.one@ift458.test",
  "password": "Coursework12345",
  "passwordConfirmation": "Coursework12345"
}
```

2. Repeat signup with the same email. Expect 409 Conflict. Try a different email with mismatched passwordConfirmation. Expect 400. Do not change stored database records to make these tests pass.

3. Expand POST /users/login and submit the registered email and password. Expect 200 with token, expiresAt, and a public user profile. The token is valid for 24 hours. Capture the response structure, but cover the complete token in submitted screenshots.

```
{
  "email": "student.one@ift458.test",
  "password": "Coursework12345"
}
```

4. Test a wrong password and an unregistered email separately. Expect 401 for both and the same message, Incorrect email or password. Explain why the two failures do not disclose which emails exist.

5. Select Authorize at the top of Swagger. Paste only the token value into bearerAuth, choose Authorize, and close the dialog. Swagger adds the Bearer prefix automatically. For a separate HTTP client, use the header shown below.

Authorization: Bearer <your token>

Understand the credential

The token is an opaque session credential, not a JWT to decode. The sessions collection stores its SHA-256 hash, user reference, and expiry. Passwords are stored as bcrypt hashes. The server checks token expiry and whether the associated user still exists on each protected request.

Do not paste tokens into URLs, query parameters, logs, or your report. Swagger Logout removes the credential from Swagger requests; this build has no server logout route, so that action does not revoke a copied token.

Evidence 2

Capture signup success, duplicate or invalid signup, login success with token covered, and both invalid-login responses. With read-only database access, confirm the user password is a bcrypt hash rather than the submitted password. Never edit the hash.

How HTTP carries authentication

Stateless HTTP and application sessions

HTTP does not automatically remember that a previous request logged in. Each protected request must supply a credential. An application can maintain sessions on top of HTTP. In this project, MongoDB holds session records, so the application uses server-side state even though HTTP is stateless. A persistent network connection does not replace the credential check.

What goes into the request

The first login request carries email and password in a JSON body. After successful verification, the server returns an opaque token. For later game and score operations, the client sends that token in the Authorization header instead of resending the password. In Swagger, paste only the token into Authorize; Swagger constructs the header.

```
GET /api/v1/games HTTP/1.1
Host: localhost:4001
Authorization: Bearer <your-token>
Accept: application/json
```

For this application, the authentication middleware hashes the submitted token, finds an unexpired session, checks that the user exists, and allows the resource handler to run. Without that proof, the response is 401. The token is a credential even though it is not the user password.

Why the header is useful and what makes transport safe

The Authorization header is the standard location for a bearer credential. It keeps the token out of the URL, where it can leak through history and URL logs. The header itself is not encrypted. HTTPS encrypts HTTP headers and bodies in transit; ordinary HTTP does not. The localhost HTTP address is for local lab use. Network-accessible or deployed use requires HTTPS.

A bearer token can be used by anyone who obtains it until it expires or is invalidated. The client developer tools, application server, and TLS-terminating proxies can see it. Do not share it, log the Authorization header, or expose it in screenshots. Clearing Swagger state does not revoke a token copied elsewhere.

Check your understanding

Explain why a successful login followed by a request without a token returns 401. Then explain why moving a password or token into a header does not make a plain HTTP connection secure. Finally, identify the server-side state used by this project.

Further reading

MDN Web Docs. Overview of HTTP, section HTTP is stateless but not sessionless.
<https://developer.mozilla.org/en-US/docs/Web/HTTP/Guides/Overview>

RFC 6750. The OAuth 2.0 Authorization Framework Bearer Token Usage, sections 2.1 and 5. <https://www.rfc-editor.org/rfc/rfc6750>

These readings explain HTTP and the bearer convention. This lab does not implement a complete OAuth authorization flow.

3 Demonstrate authenticated resource operations

Keep Swagger authorized. Create test records bearing your name so you can identify them later. Copy each returned `_id` before editing or deleting a record. All paths below are relative to `/api/v1`.

Games

```
{
  "title": "Student One Test Game",
  "developer": "Course Lab",
  "description": "Temporary record for authentication testing",
  "platform": "pc",
  "submittedBy": "Student One"
}
```

POST `/games` with this body, then GET `/games` and GET `/games/{id}`. PATCH your record with `{"description": "Updated during the lab"}`. Confirm the change with another GET. Keep the record until you finish the anonymous tests on the next page.

Scores

```
{
  "gameTitle": "Student One Test Game",
  "points": 1200,
  "playerEmail": "student.one@ift458.test",
  "platform": "pc"
}
```

POST `/scores` three times, using different points values. Expect status submitted for each new score. GET `/scores` and locate your three records; other records may also appear. GET one score by its `_id` and PATCH it with `{"points": 1300}`. Confirm the stored change.

Operation	Expected response
POST a valid game or score	201 Created
GET a list or existing record	200 OK
PATCH an existing record with valid fields	200 OK
DELETE an existing test record after Part 4	204 No Content
GET a deleted record while authorized	404 Not Found

Validate input while authorized

POST a game without title and POST a score with points set to -1. Both must return 400. A game requires title, developer, and description. A score requires `gameTitle`, an integer points value from 0 to 100000000, and a valid `playerEmail`. An unauthenticated request returns 401 before these field rules are tested.

Evidence 3

Record the method, path, actual status, and observed change for both resources. Include screenshots of creation, an updated record, and validation failures. Use only your own disposable records for changes and deletion.

4 Prove anonymous requests are rejected

1. Keep the game and score IDs from Part 3. Select Authorize, then Logout if shown, and close the dialog. Reload Swagger to clear old displayed responses. Do not authorize again yet.
2. Run every row below for both games and scores. Use existing test IDs and valid request bodies. For each row, record the actual response and whether it matches the expected 401.

Request without a token	Expected	Games actual	Scores actual
GET /resource	401	_____	_____
POST /resource with valid JSON	401	_____	_____
GET /resource/{id}	401	_____	_____
PATCH /resource/{id} with valid JSON	401	_____	_____
DELETE /resource/{id}	401	_____	_____

Replace resource with games or scores. All ten checks must be rejected. A public Swagger page does not make these API operations public.

3. In Authorize, enter invalid-token as the bearer value. Execute GET /games and GET /scores again. Both must return 401. Remove the invalid credential afterward.
4. For a browser-independent check, run the command below in another terminal. It sends no browser cookies or Authorization header. On Windows PowerShell, use curl.exe in place of curl.

```
curl -i http://localhost:4001/api/v1/games
```

5. Authorize with your valid token again. GET the saved game and score to confirm the rejected edit and delete attempts did not change or remove them. Now DELETE your disposable test records and expect 204. GET a deleted record while still authorized and expect 404.

Read the correct Swagger section

Server response shows what the server actually returned. The 200 Example Value lower on the page is generated documentation and can show a sample game even when your real request failed with 401. A visible example is not evidence of access.

Evidence 4

Submit the completed ten-check matrix plus screenshots showing anonymous GET and DELETE failures, an invalid-token failure, and successful access after reauthorization. Include the final authenticated delete response. Use request and response evidence, not the lock icon alone.

Explain the remaining limitation

This build checks who is calling but does not restrict access by role or record owner. submittedBy and playerEmail remain client-supplied values. Explain why authentication alone does not prove that a player owns a score or may verify it.

5 Reset browser state and troubleshoot

Use this reset only if old page content or an earlier Swagger authorization is confusing your test. Select Clear Swagger Data and Reload near the top of Swagger. This resets authorization, entered values, and displayed responses in this tab; it does not revoke issued tokens or delete database records. Clearing cookies alone does not prove the Authorization header is absent.

1. For localhost site data in Chrome, select the site-controls icon to the left of the address. Choose Cookies and site data, then Manage on-device site data if available. Remove the localhost entry and select Done. Labels can differ between browser versions.
2. Reload the page. To bypass cached page files, use Command Shift R on macOS or Ctrl Shift R on Windows/Linux. Execute a new request and read Server response. Deleting site data is separate from clearing all cached browser files.

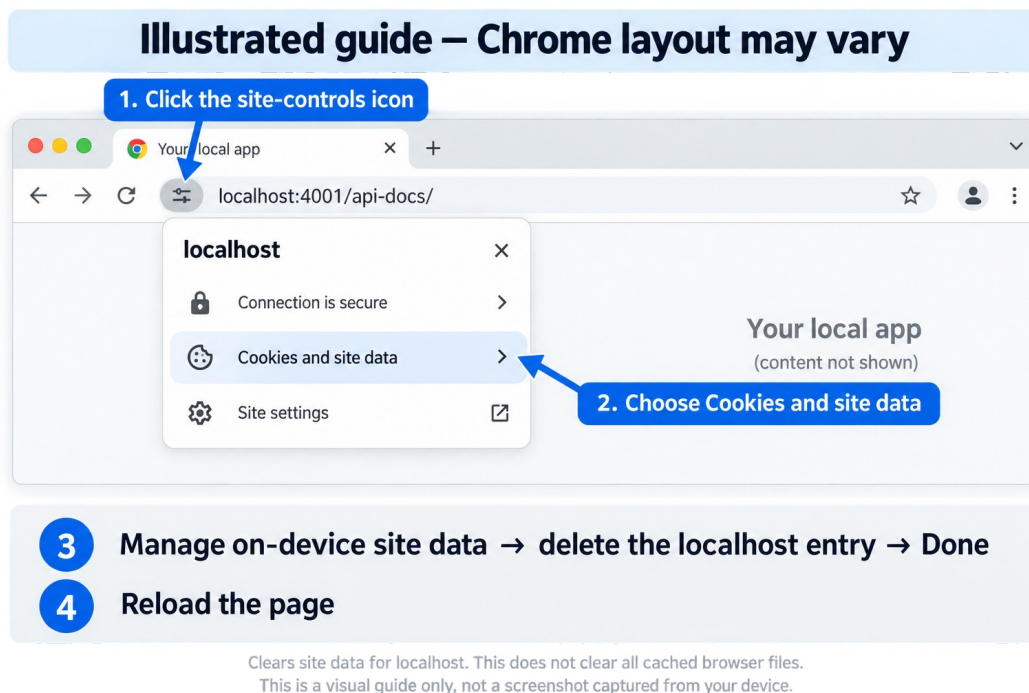


Figure 1. Illustrated Chrome navigation guide. This is not a captured browser session; labels and connection indicators may differ on your computer.

If a result does not match

401 after login: confirm the token was entered into Authorize, not just returned by login. Check expiry and whether the user still exists. Use a fresh login if necessary.

200 without an intended login: inspect the executed request for an Authorization header, confirm the URL and port, and repeat the curl check from Part 4. An old server or deployed Level 1 app can behave differently from the local Level 2 project.

Database or startup failure: read the numbered checklist in the terminal and log. Check DATABASE, the exact Atlas variable names, database-user permissions, and network access. If the shutdown summary is missing, preserve the log and record the error; do not edit protected source files to manufacture evidence.

6 Assemble and submit your evidence

Build your Word report

Use the existing coursework submission format: 12 point Times New Roman, double spacing, one-inch margins, and page numbers. Include a title page with the assignment title, your name, institution, course, instructor, and submission date. Number each screenshot as a figure, give it a descriptive title, and add a note explaining the method, path, status, and what the result proves.

Report section	Required evidence
Setup	Student identity, database startup, automated test totals
Registration and login	201 signup; signup failure; 200 login with token covered; two 401 login failures
Authenticated operations	Games and scores created, read, updated, deleted; validation 400; deleted record 404
Authentication enforcement	Ten-check matrix; anonymous GET and DELETE 401; invalid token 401; reauthorization succeeds
Analysis and log	90 to 120 word analysis; original timestamped application log

Write your Security Log Analysis

In 90 to 120 words, cite timestamps or line numbers from your own log. Identify a successful login and a rejected resource request, explain how their status codes support your conclusion, describe how credentials are protected, and identify one remaining authorization limitation. Explain any mismatch between expected and observed results. Do not invent log entries or test outcomes.

Collect the correct log

Stop the server with Ctrl+C and keep the timestamped log for the run you used as evidence. The filename records the run start, not the shutdown time. Keep the original file unchanged. Legacy L1-CLO labels and the earlier anonymous-deletion advice may still appear: assess Level 2 enforcement from actual request status codes and your results matrix, not from a legacy coverage score alone.

Package the submission

1. Save the report as LastName_FirstName_IFT458_Level2.docx. Include only your own observations and screenshots; cover passwords, bearer tokens, and database credentials.
2. Place the report and original timestamped .log file in LastName_FirstName_IFT458_Level2.zip. Do not include .env, config.env, node_modules, or database exports.
3. Upload the ZIP to the Level 2 assignment location specified by your instructor. Verify that both files are inside and the upload completed. If blocked, include the error, attempted fixes, and available log. The instructor determines credit for incomplete work.

Completion check

You can explain the token flow, reproduce 401 without a token, complete authenticated CRUD for both resources, distinguish sample documentation from live responses, and support each claim with your own evidence.

7 How passwords are hashed before saving

Read this walkthrough alongside the lab. These snapshots are extracted from the supplied source files, not from a database or a live credential. Passwords are hashed with bcrypt, not encrypted: login checks a candidate against the stored hash rather than decrypting a stored password.

```
models/userModel.js

1  const mongoose = require('mongoose');
2  const bcrypt = require('bcryptjs');
...
4  const BCRYPT_ROUNDS = Number(process.env.BCRYPT_ROUNDS) || 12;
...
49 userSchema.pre('save', async function () {
50   if (!this.isModified('password')) return;
51   this.password = await bcrypt.hash(this.password, BCRYPT_ROUNDS);
52   this.passwordConfirmation = undefined;
53 });
...
60 module.exports = mongoose.model('User', userSchema);
```

Source snapshot: `models/userModel.js`. Original line numbers are shown; wrapped lines continue the preceding line. Ellipses indicate omitted source, not executable code.

Line by line

Line 1 Loads Mongoose, which connects the schema and model to MongoDB.

Line 2 Loads `bcryptjs` for password hashing and comparison.

Line 4 Converts `BCRYPT_ROUNDS` from configuration into a number; the expression falls back to 12 when that value is falsy. This is the bcrypt cost setting, not a count of 12 simple hash operations.

Line 49 Registers an asynchronous pre-save hook. Mongoose validation runs before this hook. A normal function gives this access to the user document being saved.

Line 50 Returns early if password was not changed, preventing an unchanged stored hash from being hashed again when other fields are saved.

Line 51 Waits for `bcrypt.hash` to generate a salted password hash and replaces the document's password value with that hash. Passing a numeric cost lets bcrypt generate the salt.

Line 52 Removes `passwordConfirmation` from the document before persistence.

Line 53 Closes the hook function and its registration. Saving can continue after the awaited hash finishes.

Line 60 Compiles and exports the User model. Other modules use this model to create and query user documents.

What reaches the database

During signup, the request password temporarily exists in application memory. Before the user document is written, this hook replaces it with a hash. The bcrypt output carries information needed for later comparison, including the salt and cost. It is not a reversible encrypted copy of the password.

Library reference: bcryptjs hashing and comparison API, <https://github.com/dcodeIO/bcrypt.js>

8 How signup saves the user document

```
controllers/authController.js

3   const User = require('../models/userModel');
...
6   const publicUser = (user) => ({ id: user._id, name: user.name, email: user.email
   });
...
9   exports.signup = async (req, res) => {
...
11    const { name, email, password, passwordConfirmation } = req.body || {};
12    const user = await User.create({ name, email, password, passwordConfirmation });
...
14    res.status(201).json({ status: 'success', data: { user: publicUser(user) } });
15  };

```

Source snapshot: `controllers/authController.js`. Original line numbers are shown; wrapped lines continue the preceding line. Ellipses indicate omitted source, not executable code.

Line 3 Imports the User model, including its validation rules and save hook.

Line 6 Defines `publicUser`, which returns only the user ID, name, and email. The password hash is excluded from this response shape.

Line 9 Exports the asynchronous signup handler called for POST `/users/signup`.

Line 11 Reads only the four expected fields from the request body. The empty-object fallback avoids destructuring an absent body.

Line 12 Creates and saves a User. Mongoose validates the fields, runs the password save hook, and writes the document to MongoDB. `Await` resumes after that operation succeeds; a failure prevents the success response.

Line 14 Sends HTTP 201 and the public user profile. Neither a password nor a token is returned by signup.

Line 15 Closes the signup handler.

Password rules applied before hashing

```
models/userModel.js

21    password: {
22      type: String,
23      required: [true, 'Please provide a password.'],
24      minlength: [8, 'Password must be at least 8 characters.'],
25      select: false // never returned by queries unless asked for with
      .select('+password')
26    },

```

Source snapshot: `models/userModel.js`. Original line numbers are shown; wrapped lines continue the preceding line. Ellipses indicate omitted source, not executable code.

Line 21 Starts the password field definition.

Line 22 Declares a string field.

Line 23 Requires a password and supplies the missing-value message.

Line 24 Requires at least eight characters during schema validation.

Line 25 Excludes password from normal query results. This controls selection; it does not hash the value or guarantee that every response is safe.

Line 26 Closes the password field definition.

The adjacent passwordConfirmation rule requires confirmation when password changes and compares the two values before hashing. This checks registration input; the next section explains login verification against a stored hash.

Mongoose reference: create triggers save hooks; validation runs before user pre-save hooks.

<https://mongoosejs.com/docs/middleware.html>

9 How login validates the password

```
controllers/authController.js

18  exports.login = async (req, res) => {
19    const { email, password } = req.body || {};
...
22    if (!email || !password) {
23      return res.status(400).json({ status: 'fail', message: 'Please provide email
and password.' });
24    }
...
27    const user = await User.findOne({ email: String(email).toLowerCase()
}).select('+password');
...
30    if (!user || !(await user.isPasswordMatch(String(password)))) {
31      return res.status(401).json({ status: 'fail', message: 'Incorrect email or
password.' });
32    }
```

Source snapshot: `controllers/authController.js`. Original line numbers are shown; wrapped lines continue the preceding line. Ellipses indicate omitted source, not executable code.

Line 18 Starts the asynchronous login handler.

Line 19 Reads email and password from the JSON request body.

Line 22 Checks whether either required value is missing or falsy.

Line 23 Returns HTTP 400 and stops this handler if either value is absent.

Line 24 Closes the missing-values condition.

Line 27 Converts email into a lowercase string, queries for that user, and explicitly selects the normally hidden password hash. The string conversion prevents an email object from being passed as a MongoDB query operator here.

Line 30 Rejects an unknown user or a failed password comparison. Short-circuit evaluation skips the method call when no user exists. Await waits for the comparison result.

Line 31 Returns the same HTTP 401 message for an unknown email and an incorrect password; this response does not identify which value was wrong.

Line 32 Closes the rejection condition. Only a successful comparison reaches token creation.

The comparison method called on line 30

```
models/userModel.js

56  userSchema.methods.isPasswordMatch = function (candidatePassword) {
57    return bcrypt.compare(candidatePassword, this.password);
58  };
```

Source snapshot: `models/userModel.js`. Original line numbers are shown; wrapped lines continue the preceding line. Ellipses indicate omitted source, not executable code.

Line 56 Adds `isPasswordMatch` to User documents and accepts the submitted `candidatePassword`.

Line 57 Returns `bcrypt.compare(candidatePassword, this.password)`. It uses the stored hash to check the candidate and resolves to true or false. It does not decrypt a password.

Line 58 Closes the document method.

Schema validation checks whether new input meets rules such as minimum length. Login password verification asks whether the supplied password matches this existing account. These are different checks.

10 How a token is created and returned

This is the continuation of the login handler after the password check succeeds. The file imports `randomBytes` and `createHash` from `node:crypto` and imports the `Session` model before this code runs.

```
controllers/authController.js

34   const token = randomBytes(32).toString('hex');
35   const expiresAt = new Date(Date.now() + 24 * 60 * 60 * 1000);
36   await Session.create({
37     tokenHash: createHash('sha256').update(token).digest('hex'),
38     user: user._id,
39     expiresAt
40   });
41   res.set('Cache-Control', 'no-store').status(200).json({
42     status: 'success',
43     token,
44     expiresAt,
45     data: { user: publicUser(user) }
46   });
47   };
```

Source snapshot: `controllers/authController.js`. Original line numbers are shown; wrapped lines continue the preceding line. Ellipses indicate omitted source, not executable code.

Line 34 Generates 32 cryptographically random bytes and encodes them as a 64-character hexadecimal token. The token is opaque; it is not a JWT or an encoded password.

Line 35 Calculates an expiry 24 hours after the current time. Multiplying by 1000 converts seconds to milliseconds.

Line 36 Begins creating and awaiting a session document in MongoDB.

Line 37 Hashes the random token with SHA-256 and stores the hexadecimal digest as `tokenHash`. The usable raw token is not stored in this session document.

Line 38 Associates the session with the authenticated user's MongoDB ID.

Line 39 Adds `expiresAt` using property shorthand, equivalent to `expiresAt: expiresAt`.

Line 40 Closes and awaits the `Session.create` call before sending success.

Line 41 Sets `Cache-Control` to `no-store` and prepares an HTTP 200 JSON response. This cache directive is not encryption.

Line 42 Sets the response status label to success.

Line 43 Returns the raw token to the client using property shorthand. The client needs this value for subsequent requests.

Line 44 Returns the expiry timestamp, which JSON serializes as a date string.

Line 45 Returns the public user profile without the password hash.

Line 46 Closes and sends the JSON response.

Line 47 Closes the login handler.

The password hash and `tokenHash` have different jobs: `bcrypt` checks a human-chosen password; SHA-256 identifies a high-entropy random token in the sessions collection. The token is returned in the login response body, not automatically attached to later requests by the server.

11 How the client adds the token to a header

The client must attach the credential to each protected request because HTTP does not automatically remember a previous login. Swagger uses the security configuration below to build that request header after the user selects Authorize.

```
SwaggerTFT458.js
69     security: [{ bearerAuth: [] }],
...
241     securitySchemes: { bearerAuth: { type: 'http', scheme: 'bearer', description:
      'Opaque session token returned by login; expires after 24 hours.' } },
```

Source snapshot: SwaggerTFT458.js. Original line numbers are shown; wrapped lines continue the preceding line. Ellipses indicate omitted source, not executable code.

Line 69 Declares bearerAuth as the default security requirement for documented operations. The empty array contains no OAuth scopes; it does not mean anonymous access. Signup and login override this requirement with security: [].

Line 241 Defines bearerAuth as an HTTP bearer scheme. Swagger uses this definition to show the token input and prefix requests with Bearer. The description explains the token lifetime. OpenAPI describes authentication; server middleware actually enforces it.

The same header in the supplied REST Client example

```
requests.http
46  GET {{host}}/games
47  Authorization: Bearer {{login.response.body$.token}}
```

Source snapshot: requests.http. Original line numbers are shown; wrapped lines continue the preceding line. Ellipses indicate omitted source, not executable code.

Line 46 Sends GET to the games endpoint. The host variable is defined earlier in requests.http.

Line 47 In the VS Code REST Client, reads token from the named login request's JSON response and inserts it after Bearer. Run the named login request first. This templating is a client feature, not JavaScript executed by the server.

Follow the token from response to request

- Execute POST /users/login and obtain token from the successful JSON response.
- In Swagger, select Authorize, paste only the token, apply it, and close the dialog.
- Execute GET /games. Swagger adds Authorization: Bearer followed by that token.
- Read Server response for the real result. Without a valid token, the server returns 401.

Using the header keeps the credential out of the URL. It does not encrypt it. HTTPS protects headers and bodies in transit; plain HTTP does not. Treat the token as a secret and cover it in screenshots. Use Clear Swagger Data and Reload to clear this tab's authorization; that action does not invalidate a copied token.

Protocol reference: RFC 6750, sections 2.1 and 5, <https://www.rfc-editor.org/rfc/rfc6750>

12 How the server verifies the header

The authentication middleware imports `createHash`, `Session`, and `User`. `app.js` runs this middleware before the `game` and `score` routers. This completes the path from password validation to access on the next request.

```
middleware/authenticate.js

5  module.exports = async (req, res, next) => {
6    const match = /^Bearer ([a-f0-9]{64})$/i.exec(req.get('authorization') || '');
7    const deny = () => res.set('WWW-Authenticate', 'Bearer').status(401).json({
8      status: 'fail', message: 'Authentication required. Log in and provide a valid
bearer token.'
9    });
10   if (!match) return deny();
...
12   const tokenHash = createHash('sha256').update(match[1]).digest('hex');
...
14   const session = await Session.findOne({ tokenHash, expiresAt: { $gt: new Date() }
});
15   if (!session) return deny();
16   const user = await User.findById(session.user);
17   if (!user) return deny();
18   req.user = user;
19   next();
20  };
```

Source snapshot: `middleware/authenticate.js`. Original line numbers are shown; wrapped lines continue the preceding line. Ellipses indicate omitted source, not executable code.

Line 5 Exports an async Express middleware function; next passes control onward.

Line 6 Reads Authorization and matches Bearer followed by one space and 64 hexadecimal characters. `match[1]` contains the token; a missing header becomes an empty string.

Line 7 Defines `deny`, which sets a Bearer challenge and HTTP 401 before returning JSON.

Line 8 Provides the failure status and login guidance in that JSON body.

Line 9 Closes the `deny` helper.

Line 10 Immediately rejects a missing or malformed credential.

Line 12 Computes the same SHA-256 token digest used when the session was created.

Line 14 Finds a matching session whose expiry is later than now. Expiry is enforced immediately, independently of delayed database TTL cleanup.

Line 15 Rejects an unknown or expired session.

Line 16 Loads the user referenced by the session.

Line 17 Rejects a token whose user has been deleted.

Line 18 Attaches the verified user document to this request as `req.user`.

Line 19 Invokes the next middleware or resource handler after all checks pass.

Line 20 Closes the authentication middleware.

Authentication establishes the caller's identity. This build does not yet enforce resource ownership or roles; a valid session alone allows the `game` and `score` operations described in the lab.